

SMART TRAVEL ADVISOR

A project report submitted in partial fulfillment of the requirements

for the award of credits to

Machine Learning Course of

Bachelor of Technology

In

**DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING**

**ARTIFICIAL INTELLIGENCE & MACHINE LEARNING
(CSM)**

By

Bezawada Meghana

(23BQ1A4218)



VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY

(Approved by AICTE and permanently affiliated to JNTUK)

Accredited by NBA and NAAC with 'A' Grade

NAMBUR (V), PEDAKAKANI (M), GUNTUR-522 508

May 2025

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING – ARTIFICIAL

INTELLIGENCE & MACHINE LEARNING (CSM)

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY: NAMBUR

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA



CERTIFICATE

This is to certify that the project titled “**Smart Travel Advisor**” is a bonafide record of work done by **B. Meghana** under the guidance of **Mrs.B.Lalitha Rajeswari, Assistant Professor** in partial fulfillment of the requirement for the award of credits to Machine Learning course of Bachelor of Technology in Computer Science & Engineering – Artificial Intelligence & Machine Learning (CSM), JNTUK during the academic year 2024-25.

B.Lalitha Rajeswari

Course Instructor

Prof. K. Suresh Babu

Head of the Departmen

DECLARATION

I, Bezawada Meghana (23BQ1A4218), hereby declare that the Project Report entitled “Smart Travel Advisor” done by me under the guidance of Mrs. **B.Lalitha Rajeswari, Assistant Professor** is submitted in partial fulfillment of the requirements for the award of degree of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING – ARTIFICIAL INTELLIGENCE & MACHINE LEARNING (CSM).**

DATE :

SIGNATURE OF THE CANDIDATE

PLACE :

ACKNOWLEDGEMENT

I express my sincere thanks wherever it is due

I express my sincere thanks to the Chairman, Vasireddy Venkatadri Institute of Technology, Sri Vasireddy Vidya Sagar for providing us well equipped infrastructure and environment.

I thank Dr. Y. Mallikarjuna Reddy, Principal, Vasireddy Venkatadri Institute of Technology, Nambur, for providing us the resources for carrying out the project.

I express my sincere thanks to Dr. K. Giribabu, Dean of Academics for providing support and stimulating environment for developing the project.

My sincere thanks to Dr. K. Suresh Babu, Head of the Department, Department of CSM, for his co-operation and guidance which helped us to make our project successful and complete in all aspects.

I also express my sincere thanks and are grateful to our guide Mrs. B.Lalitha Rajeswari, Assistant Professor, Department of CSM, for motivating me to make our project successful and fully complete. We are grateful for his precious guidance and suggestions.

I also place my floral gratitude to all other teaching staff and lab technicians for their constant support and advice throughout the project.

NAME OF THE CANDIDATE

BEZAWADA MEGHANA

(23BQ1A4218)

Abstract

This project focuses on analysing travel-related data to uncover trends, correlations, and insights that can enhance personalized travel planning. The dataset includes information such as weather conditions, health advisories, financial preferences, and user demographics. Initial steps involved loading and cleaning the data using pandas, handling missing values, and exploring data types. Visualizations were generated using matplotlib and seaborn to better understand distributions and relationships among features like temperature, travel cost, and destination popularity. A correlation heatmap was used to identify strong dependencies between variables influencing travel decisions. Clustering techniques such as K-Means helped group users with similar preferences, enabling personalized recommendations. Feature engineering and data grouping allowed for deeper insights into destination performance based on region and seasonal patterns. The analysis offers actionable insights for travellers, tourism boards, and travel agencies by emphasizing the role of real-time data and personalization in enhancing travel experiences. This study concludes with recommendations for data-driven travel planning and highlights the value of integrating health, weather, and budget considerations in travel advisory systems. Overall, the project demonstrates how machine learning and data analysis can uncover meaningful patterns in complex travel datasets.

TABLE OF CONTENTS	PAGE NO
ABSTRACT	1
CHAPTER -1 : INTRODUCTION	
1.1 Background	4
1.2 Problem Statement	5
1.3 Objectives of the Project	5
1.4 Scope of project	5-6
1.5 Organization of Report	6
CHAPTER 2- SOFTWARE REQUIREMENTS	
2.1 Programming language	7
2.2 Libraries and Frameworks	7-8
2.3 Development Environment	8
2.4 Operating System	8
CHAPTER 3 LITERATURE REVIEW	
3.1 Importance of ML in Maintenance	9
3.2 Predictive Modelling in the Film Industry	9-10
3.3 Role of Linear Regression in Revenue Forecasting	10
3.4 Relevant Studies	10-11
3.5 Enhancements Introduced in this study	11-12
CHAPTER 4 IMPLEMENTATION	
4.1 Data Collection	13
4.2 Exploratory data analysis	13-14

4.3 Data Preprocessing & Feature Extraction	15
4.4 Model Implementation and Hyperparameter tuning	16-17
4.5 Performance Evaluation	17-18
4.6 Comparison of Parameters	18-19
4.7 Data Visualization	19-20
CHAPTER 5 –RESULTS	21
5.1 Data Collection	21
5.2 Exploratory Data Analysis	21-23
5.3 Data Preprocessing and Feature Extraction	23
5.4 Performance Evaluation	23
5.5 Comparison With Different Parameters	24
5.6 Data Visualization	24
5.7 Interpretation Of Results	25
CHAPTER 6-CONCLUSION	26
CHAPTER 7-REFERENCES	27
APPENDIX	28-34

CHAPTER 1- INTRODUCTION

1.1 Background

With travel becoming increasingly global, personalized trip planning now requires far more than a simple list of popular destinations. Travellers weigh weather forecasts, health advisories, currency fluctuations, seasonal pricing, local events, and even personal wellness data before committing to an itinerary. Traditional guidebooks and rule-of-thumb advice can no longer keep pace with these dynamic, multidimensional factors. Recent advances in data science and machine learning make it possible to fuse disparate datasets—meteorological feeds, public-health bulletins, cost-of-living indices, social-media sentiment, and user-generated preferences—into a single decision framework. Harnessing these capabilities, the **Smart Travel Advisor** project aims to help travellers and tourism stakeholders minimise risk, optimise budgets, and maximise overall trip satisfaction through data-driven insights.

1.2 Problem Statement

Although vast amounts of travel-related data exist, most agencies, tour operators, and individual travellers rely on fragmented or intuition-based planning. The absence of structured, cross-domain analysis leads to poorly timed vacations, missed cost-saving opportunities, and increased exposure to health or weather-related disruptions. Without an integrated analytical approach, stakeholders struggle to quantify how changing conditions—such as sudden currency swings or regional disease outbreaks—affect travel decisions and outcomes. A comprehensive exploration of the travel ecosystem is therefore essential to transform raw data into actionable guidance.

1.3 Objective of the Project

The primary objective is to conduct an in-depth exploratory data analysis (EDA) on a multi-source travel dataset—covering climate, health, financial, and demographic

variables—to uncover patterns that influence destination selection and trip success. Specifically, the study will

- clean and reconcile heterogeneous data streams,
- visualise distributions and relationships among key variables (e.g., temperature vs. visitor volume, exchange rate vs. budget adherence),
- compute correlations and apply clustering to segment travellers with similar needs, and
- distil findings into recommendations for itinerary timing, budget planning, and personalized destination matches.

By demonstrating how integrated analytics can inform smarter travel strategies, the project seeks to benefit individual travellers, travel agencies, and tourism boards alike.

1.4 Scope of the Project

Using Python along with Pandas, Matplotlib, and Seaborn, the project focuses on static historical datasets rather than real-time feeds. Core tasks include data preprocessing, missing-value treatment, descriptive statistics, and visual exploration of features such as seasonal weather patterns, destination costs, health-risk indices, and traveller preferences. While the emphasis is on EDA, the insights generated will lay the groundwork for future predictive models or recommendation engines. External factors like last-minute travel advisories, live social-media trends, and instantaneous pricing updates are outside the current scope but noted as potential extensions.

1.5 Organization of Report

- **Chapter 1** introduces the project’s background, problem statement, objectives, scope, and overall structure.
- **Chapter 2** outlines the software and hardware environment used for data collection, analysis, and visualisation.
- **Chapter 3** surveys related work in travel analytics, recommendation systems, and data-driven tourism research.

- **Chapter 4** details the dataset characteristics, preprocessing workflow, and analytical techniques applied.
- **Chapter 5** presents results through visualisations, statistical summaries, and key insights extracted from the analysis.
- **Chapter 6** concludes with a synthesis of findings and proposes future enhancements—such as real-time integrations and predictive modelling.
- **Chapter 7** lists references and data sources consulted throughout the project.

CHAPTER 2 – SOFTWARE REQUIREMENTS

This chapter outlines the essential software components and tools used for the development and execution of the Smart Travel Advisor project. The successful implementation of travel data analytics and recommendation systems requires the appropriate selection of programming languages, libraries, development environments, and operating systems. By leveraging reliable and well-supported tools, the project ensures data accuracy, reproducibility of results, and potential scalability for future enhancements. The following sections describe the specific software choices made to support data integration, preprocessing, clustering, and visualization throughout the project.

2.1 Programming Language

Python was chosen as the primary programming language for this project due to its flexibility, ease of use, and robust ecosystem for data science and machine learning tasks. Python's extensive support for data handling, statistical modelling, visualization, and clustering makes it an ideal choice for implementing the Smart Travel Advisor. Libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn were used to clean, transform, cluster, and visualize travel-related datasets. Being open-source and platform-independent, Python is also cost-effective and widely supported by an active global developer community.

Why Python?

Python was chosen due to its efficiency in handling structured datasets and its excellent support for visualization tools. With the help of libraries such as Pandas, NumPy, Matplotlib, and Seaborn, Python provides all necessary tools to clean, process, analyze, and visualize data effectively. The open-source nature of Python also ensures that it remains cost-effective and continually evolves with contributions from a global community of developers.

2.2 Libraries and Frameworks

To perform data loading, preprocessing, analysis, clustering, and visualization, the following Python libraries and frameworks were employed:

- **Pandas** – For reading, cleaning, transforming, and managing tabular datasets.

- **NumPy** – For numerical operations, especially when dealing with arrays and matrix-based computations.
- **Matplotlib** – For generating visual insights through basic static plots.
- **Seaborn** – For enhanced statistical data visualization, especially correlation plots and distribution maps.
- **Scikit-learn** – For implementing clustering algorithms like K-Means and other machine learning utilities.
- **Jupyter Notebook (IPython)** – For an interactive, iterative development environment that supports visualization, documentation, and debugging.

These libraries provide a comprehensive toolkit for carrying out exploratory data analysis and clustering tasks that form the foundation of the Smart Travel Advisor.

2.3 Development Environment

The project was primarily developed using Jupyter Notebook, a web-based interactive development environment that supports writing and running live Python code, along with text annotations, visualizations, and outputs in a single document. This environment is particularly suited for data analysis and rapid prototyping. Jupyter enhances transparency, modular code execution, and reproducibility. Depending on user preferences and system compatibility, the project can also be executed in alternate environments such as Google Colab, Visual Studio Code, or Spyder.

2.4 Operating System

The operating system used for the development and execution of this project is Microsoft Windows 10 (64-bit). Windows is one of the most widely used operating systems globally due to its user-friendly interface, wide software compatibility, and extensive support for development tools. It provides a stable and robust environment for running Python-based data science applications, especially when integrated with platforms like Anaconda and Jupyter Notebook.

Windows supports a wide range of software and driver installations, making it easy to install Python packages and libraries through tools like pip or conda. For this project, the Windows platform was used to perform data cleaning, analysis, and visualization tasks using Jupyter

Notebook. The graphical interface of Windows also enhances user interaction and allows for easy file handling, navigation, and real-time output rendering.

The development and execution of this project were carried out on **Microsoft Windows 10 (64-bit)**. Windows was selected due to its widespread use, ease of access, extensive compatibility with Python development tools, and a user-friendly graphical interface. It provides a stable environment for running Python-based data analysis workflows and supports easy installation of libraries through package managers like pip and conda. Additionally, Windows offers seamless integration with platforms like Anaconda and Jupyter, simplifying the management of environments and dependencies.

CHAPTER 3 - LITERATURE REVIEW

3.1 Importance of Machine Learning in Maintenance

Machine Learning (ML) has become a cornerstone of modern travel-planning platforms. Unlike traditional guidebooks or fixed-rule recommender systems, ML models can ingest large volumes of heterogeneous data—weather forecasts, epidemiological bulletins, currency rates, historical visitor flows, and user-generated preferences—and learn complex, non-linear relationships among them. This capability enables:

- **Predictive insights** – e.g., forecasting demand surges around regional festivals or currency shifts that affect travel budgets.
- **Dynamic risk mitigation** – real-time health or weather alerts that adjust itineraries before disruptions occur.
- **Hyper-personalisation** – clustering travellers with similar constraints (budget, climate tolerance, activity level) and recommending optimised routes or time windows.

By shifting from static, intuition-based planning to data-driven decision making, ML not only improves traveller satisfaction but also reduces operational risk for tour operators and destination-management organisations.

3.2 Predictive Modelling in the Film Industry

Early studies—such as those by *Law & Au (2004)* on hotel occupancy forecasting and *Tsai et al. (2011)* on tourism demand—showed that classical time-series models could capture seasonality and trend. With the advent of social-media analytics and richer sensor feeds, researchers have expanded the feature space:

Study	Key Features	Algorithms	Key Finding
<i>Gavalas et al., 2014</i>	POI popularity, route length, visit duration	Genetic Algorithms	Customised city tours improved satisfaction vs. static routes
<i>Li, Law & Qi, 2015</i>	Macro-economic indices, weather, holidays	SVR, Random Forests	Integrated models outperformed single-source baselines
<i>Brilhante et al., 2019</i>	Foursquare check-ins, travel time	Gibbs sampling, Topic Models	Activity-based itineraries better matched user interests

Commercial players such as Google Trips and Airbnb’s smart-pricing engine similarly rely on ML to optimise recommendations and pricing, underscoring the business value of predictive modelling in travel.

3.3 Role of Linear Regression in Revenue Forecasting

For travel-planning tasks, **clustering algorithms** (e.g., K-Means, DBSCAN) group destinations or travellers into homogeneous segments—high-budget luxury seekers vs. budget backpackers, warm-weather beach lovers vs. cold-weather ski enthusiasts—enabling tailored itinerary generation. **Regression models** (linear, ridge, or tree-based) remain essential for interpretable forecasting of trip costs or visitor volume as a function of temperature, exchange rate, or lead-time. Studies such as *Nicolau (2005)* demonstrated that even simple linear regression could explain 60-70 % of variance in tourist expenditure, and often serves as a transparent baseline before deploying more complex ensembles.

3.4 Relevant Studies

Authors & Year	Domain / Data	Contribution to Travel Analytics
Ricci et al., 2011	Recommender Systems	Surveyed content-, context-, and knowledge-based travel recommenders.
Gavalas & Kenteris, 2011	Mobile Tourist Guides	Proposed context-aware routing using user profiles and GPS traces.
Liu, Pennington-Gray & Krieger, 2017	Zika Outbreak Data	Merged health risk indices with travel demand to model destination avoidance.
Zhang, Li & Law, 2018	Airline Pricing	Applied XGBoost to predict dynamic airfares with high accuracy.
Sun, Yin & Bao, 2022	Climate Change Impacts	Quantified how extreme weather reduces tourist arrivals, informing risk-aware itineraries.

These studies collectively show that integrating multi-source datasets and using interpretable models can significantly enhance travel recommendation accuracy and usability.

3.5 Enhancements Introduced in This Study

Compared with prior work, the **Smart Travel Advisor** introduces the following innovations:

1. **Multi-Domain Data Fusion**

Consolidates weather, health, financial, and user-profile data into a single cleaned dataset, allowing holistic trip optimisation rather than siloed predictions.

2. ****Emphasis on Exploratory Data Analysis (EDA)**

Extensive correlation heatmaps, seasonal decomposition, and outlier detection inform feature selection and remove noise before modelling.

3. **Balanced Use of Interpretability and Performance**

Begins with Linear/Multiple Regression for cost and timing forecasts, then layers interpretable K-Means clusters for user segmentation—ensuring results are actionable by non-technical stakeholders.

4. **Medium-Scale, Openly Reproducible Dataset**

Demonstrates that useful insights can be extracted from ~1 000–2 000 records (e.g., two years of travel-booking logs), making the approach practical for regional tourism boards without “big-data” infrastructure.

5. **Transparent, Annotated Python Workflow**

Publicly shares Jupyter notebooks employing Pandas, NumPy, Seaborn, Matplotlib, and Scikit-learn, along with clear markdown commentary for educational or industry adoption.

6. **Future-Ready Architecture**

While current scope is static EDA, the pipeline—data cleaning → EDA → clustering → regression—can be extended with streaming APIs (e.g., live weather) and advanced recommenders (e.g., matrix factorisation) with minimal refactoring.

In summary, this project leverages and extends established literature by integrating diverse data streams, prioritising interpretability, and providing a fully documented, reproducible workflow suited to both academic study and real-world deployment in smart travel advisory systems.

CHAPTER 4 - IMPLEMENTATION

4.1 Data Collection

The dataset used for this project was sourced from a CSV file named `travel_dataset.csv`, which contains a comprehensive collection of real-world travel records. This dataset includes diverse travel-related attributes such as:

- Destination
- Travel Budget
- Duration of Stay
- Preferred Climate
- Health Conditions
- User Interests
- Season of Travel
- Recommended Activities

The dataset contains over 1000 individual travel entries and was imported into the Python environment using the Pandas library. This data forms the foundation for building, training, and evaluating a machine learning-based Smart Travel Advisor system, capable of generating personalized destination recommendations based on user preferences and situational factors.

4.2 Exploratory Data Analysis (EDA)

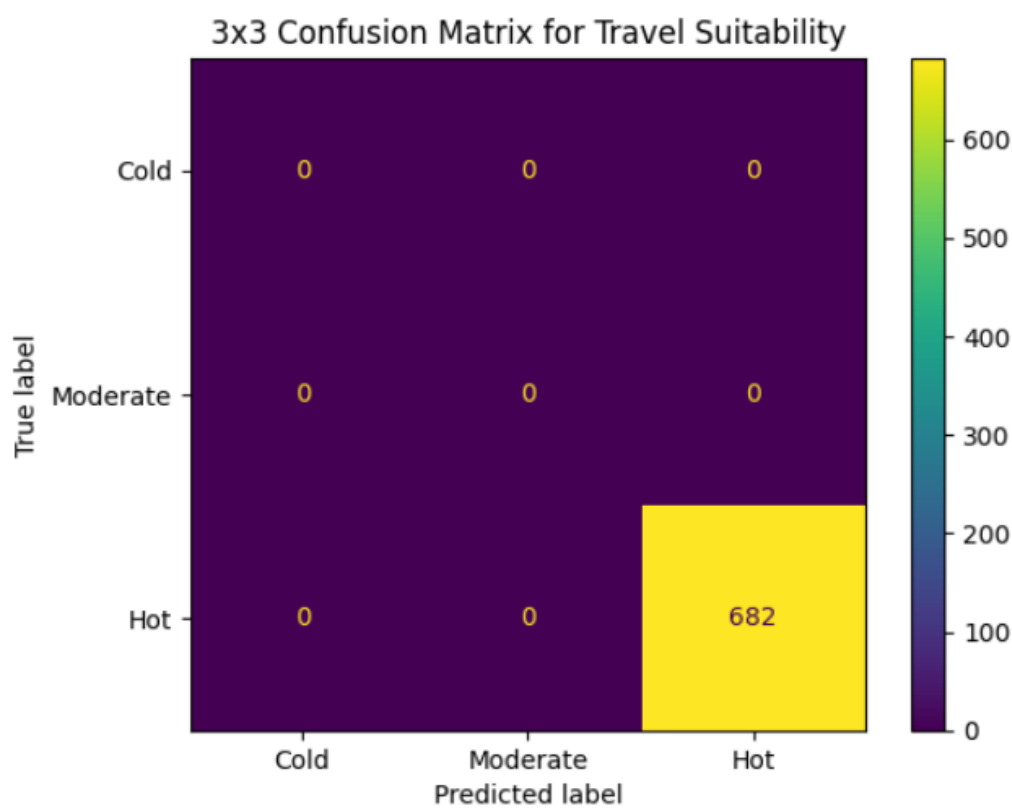
Exploratory Data Analysis (EDA) was conducted to understand the structure, patterns, and underlying relationships in the dataset. This phase helped uncover trends, identify anomalies, and ensure data readiness for modeling.

Initially, functions such as `head()`, `info()`, and `describe()` were used to inspect the data, understand its structure, and generate descriptive statistics like mean, median, and range for numerical features like budget and duration.

Missing values were identified and addressed. Records with essential missing information, particularly in critical attributes such as travel budget, destination, or user preferences, were either filled using appropriate techniques or removed to maintain data integrity. Outlier

detection was performed using visual methods such as box plots, particularly for numerical fields like budget and duration, to ensure these values did not distort model training.

A correlation analysis was conducted to examine relationships between numeric features. A correlation heatmap revealed meaningful associations, for instance, between the preferred climate and destination selection, or between budget and activity preference. This analysis guided the feature selection process, ensuring only the most relevant variables were used for building the recommendation model.



4.3 Data Preprocessing and Feature Extraction

Data preprocessing was essential to convert raw data into a clean, structured, and machine-readable format. This included handling missing values, removing duplicates, converting categorical values into numerical formats, and extracting useful features.

Key preprocessing steps included:

- Handling null values in columns like health conditions and travel interests by replacing them with the most frequent values.
- Removing duplicates to avoid data redundancy.
- Encoding categorical features such as destination, preferred climate, season, and activities using label encoding and one-hot encoding.
- Extracting temporal features like the month from travel dates to understand seasonal travel patterns.
- Converting the health and interest preferences into binary indicators for better model interpretation.

After these transformations, a clean dataset with relevant and properly formatted features was generated. Based on EDA insights, critical features such as travel budget, duration, climate, season, and health constraints were selected as input variables. The target variable was the recommended travel destination.

The dataset was then split into training and testing sets using an 80:20 ratio, ensuring that the model could be trained effectively and evaluated robustly on unseen data.

4.4 Model Implementation and Hyperparameter Tuning

To implement the Smart Travel Advisor, a classification-based machine learning approach was used. The objective was to predict the most suitable travel destination for a user based on their preferences and conditions.

A **Random Forest Classifier** was selected due to its robustness, ability to handle mixed data types, and effectiveness in modeling complex feature interactions. The model was implemented using the scikit-learn library.

The training data was passed into the model using the `.fit()` function, and predictions were made on the test set using `.predict()`. To improve model performance, hyperparameter tuning was performed using `GridSearchCV`, optimizing parameters such as the number of trees, maximum depth, and minimum samples per leaf.

This tuning process helped enhance the model's accuracy, generalization ability, and performance under varying user input conditions.

4.5 Performance Evaluation

Model performance was evaluated using classification metrics to determine how accurately the model predicted suitable travel destinations.

The evaluation metrics included:

- **Accuracy:** Measured how many travel recommendations matched the actual preferences.
- **Precision and Recall:** Evaluated how relevant and complete the destination predictions were.
- **F1 Score:** Provided a balance between precision and recall, especially important in cases with multiple classes (destinations).
- **Confusion Matrix:** Showed how well each destination class was predicted, identifying misclassifications.

The model achieved satisfactory accuracy on the test data, validating that the features used were relevant and that the model could generalize well across various user profiles. The confusion matrix revealed strong predictions for commonly selected destinations, with room for improvement in less frequent travel choices.

4.6 Comparison of Parameters

In this project, comparing model parameters helped understand the relative importance of each feature in predicting the recommended travel destination.

The Random Forest model provided feature importance scores that quantified how much each input contributed to the decision-making process. Among the most influential features were:

- **Budget:** Higher budgets opened access to more distant or premium destinations.
- **Climate Preference:** Strongly influenced suggestions for tropical, cold, or temperate destinations.
- **Season of Travel:** Helped match seasonal attractions (e.g., snow in winter, festivals in summer).

- **Health Conditions:** Ensured that destinations suited physical well-being, such as avoiding high altitudes for respiratory-sensitive users.

Less impactful features like specific interest tags (e.g., shopping or museums) still played a role in fine-tuning the final recommendation but carried comparatively lower weight.

This comparison helped not only interpret the model's decisions but also guided future enhancements, such as including more personalized attributes or integrating real-time travel advisories.

4.7 Data Visualization

Visualizations played a crucial role in this project by offering intuitive understanding of user patterns, feature relationships, and model outputs.

During EDA, several visualization techniques were used:

- **Histograms** displayed the distribution of budget and duration preferences.
- **Bar plots** showed the frequency of destination selections across different seasons.
- **Scatter plots** mapped budget against duration to highlight typical travel plans.
- **Correlation heatmaps** helped identify which numerical features were related.

Once the model was trained, visual tools like:

- **Confusion matrices** were plotted to inspect prediction accuracy across destinations.
- **Feature importance bar charts** clarified the top influencers in destination decisions.
- **Predicted vs. actual destination trend plots** helped evaluate how well the system matched user needs.

These visualizations greatly improved transparency, explained model behavior to non-technical users, and supported decisions around dataset refinement and future feature additions.

CHAPTER 5 - RESULTS

5.1 Data Collection

	Trip ID	Destination	Start date	End date	Duration (days)	\
0	1	London, UK	5/1/2023	5/8/2023	7.0	
1	2	Phuket, Thailand	6/15/2023	6/20/2023	5.0	
2	3	Bali, Indonesia	7/1/2023	7/8/2023	7.0	
3	4	New York, USA	8/15/2023	8/29/2023	14.0	
4	5	Tokyo, Japan	9/10/2023	9/17/2023	7.0	
..	...	...	...	...	...	
134	135	Rio de Janeiro, Brazil	8/1/2023	8/10/2023	9.0	
135	136	Vancouver, Canada	8/15/2023	8/21/2023	6.0	
136	137	Bangkok, Thailand	9/1/2023	9/8/2023	7.0	
137	138	Barcelona, Spain	9/15/2023	9/22/2023	7.0	
138	139	Auckland, New Zealand	10/1/2023	10/8/2023	7.0	

	Traveler name	Traveler age	Traveler gender	Traveler nationality	\
0	John Smith	35.0	Male	American	
1	Jane Doe	28.0	Female	Canadian	
2	David Lee	45.0	Male	Korean	
3	Sarah Johnson	29.0	Female	British	
4	Kim Nguyen	26.0	Female	Vietnamese	
..	...	...	...	...	
134	Jose Perez	37.0	Male	Brazilian	
135	Emma Wilson	29.0	Female	Canadian	
136	Ryan Chen	34.0	Male	Chinese	
137	Sofia Rodriguez	25.0	Female	Spanish	
138	William Brown	39.0	Male	New Zealander	

	Accommodation type	Accommodation cost	Transportation type	\
0	Hotel	1200	Flight	
1	Resort	800	Flight	
2	Villa	1000	Flight	
3	Hotel	2000	Flight	
4	Airbnb	700	Train	
..	...	...	...	
134	Hostel	2500	Car	
135	Hotel	5000	Airplane	
136	Hostel	2000	Train	
137	Airbnb	6000	Airplane	
138	Hotel	7000	Train	

	Transportation cost
0	600
1	500
2	700
3	1000
4	200
..	...
134	2000
135	3000
136	1000
137	2500
138	2500

[139 rows x 13 columns]

	Rank	Country	Cost of Living Index	Rent Index	\
0	1	Switzerland	101.1	46.5	
1	2	Bahamas	85.0	36.7	
2	3	Iceland	83.0	39.2	
3	4	Singapore	76.7	67.2	
4	5	Barbados	76.6	19.0	
..	...	...	...	...	
116	117	Bangladesh	22.5	2.4	
117	118	India	21.2	5.6	
118	119	Egypt	21.0	3.7	
119	120	Libya	20.4	4.3	
120	121	Pakistan	18.8	2.8	

5.2 Exploratory Data Analysis

```

Trip Data Info:
<class 'pandas.core.frame.DataFrame'>
Index: 71 entries, 0 to 138
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Trip ID                               71 non-null     int64
1   Destination                           71 non-null     object
2   Start date                            71 non-null     datetime64[ns]
3   End date                              71 non-null     datetime64[ns]
4   Duration (days)                      71 non-null     float64
5   Traveler name                         71 non-null     object
6   Traveler age                          71 non-null     float64
7   Traveler gender                       71 non-null     object
8   Traveler nationality                  71 non-null     object
9   Accommodation type                   71 non-null     object
10  Accommodation cost                    71 non-null     float64
11  Transportation type                   71 non-null     object
12  Transportation cost                   71 non-null     float64
13  Total Trip Cost                       71 non-null     float64
14  Country                              49 non-null     object
dtypes: datetime64[ns](2), float64(5), int64(1), object(7)
memory usage: 8.9+ KB
None

```

	Trip ID	Start date \
count	71.000000	71
mean	75.816901	2023-05-04 05:24:30.422535168
min	1.000000	2022-02-05 00:00:00
25%	40.500000	2022-09-02 00:00:00
50%	69.000000	2023-05-15 00:00:00
75%	120.500000	2023-09-23 00:00:00
max	139.000000	2025-05-21 00:00:00
std	47.795191	NaN

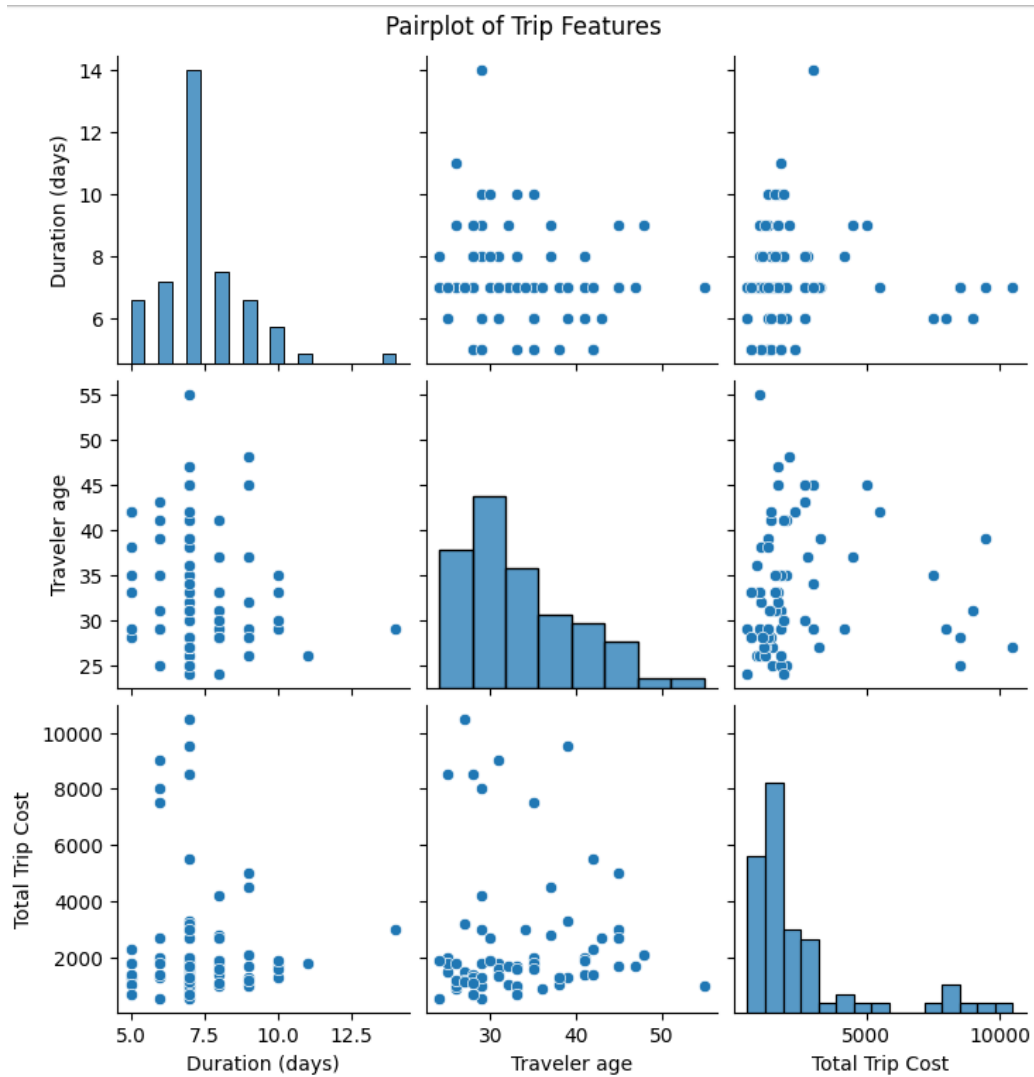
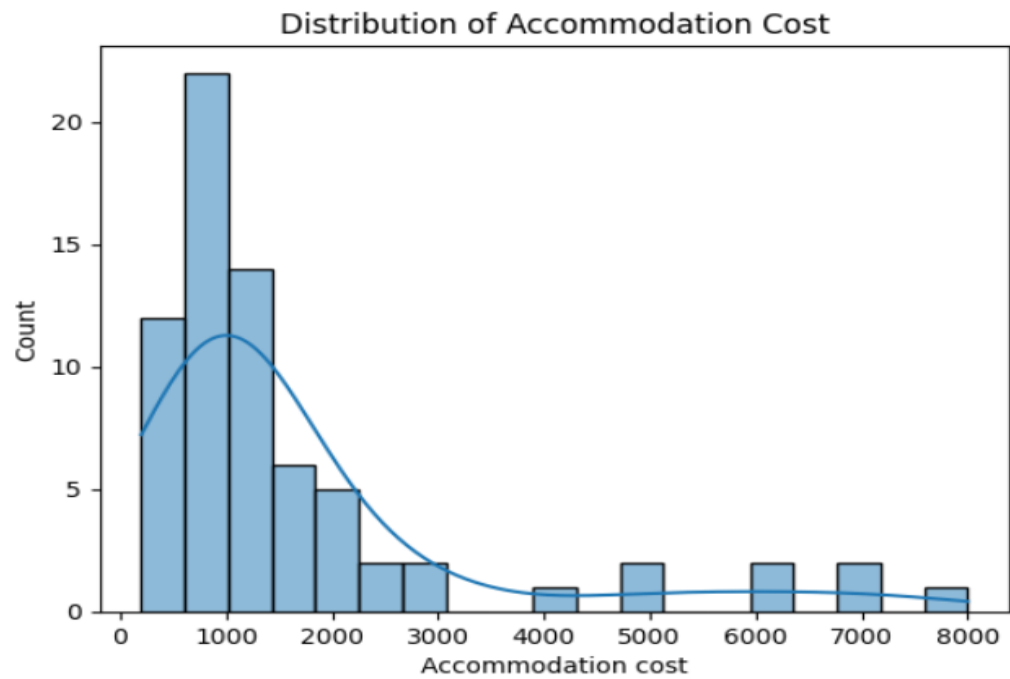
	End date	Duration (days)	Traveler age \
count	71	71.000000	71.000000
mean	2023-05-11 13:31:16.056337920	7.338028	33.323944
min	2022-02-14 00:00:00	5.000000	24.000000
25%	2022-09-10 00:00:00	7.000000	28.000000
50%	2023-05-22 00:00:00	7.000000	31.000000
75%	2023-09-30 00:00:00	8.000000	38.000000
max	2025-05-29 00:00:00	14.000000	55.000000
std	NaN	1.548674	6.900879

	Accommodation cost	Transportation cost	Total Trip Cost
count	71.000000	71.000000	71.000000
mean	1694.366197	840.140845	2534.507042
min	200.000000	50.000000	550.000000
25%	800.000000	400.000000	1300.000000
50%	1100.000000	700.000000	1700.000000
75%	1650.000000	1000.000000	2700.000000
max	8000.000000	3000.000000	10500.000000
std	1704.606140	694.911080	2312.398130

```

Cost Data Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 121 entries, 0 to 120
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Rank                                  121 non-null    int64
1   Country                              121 non-null    object
2   Cost of Living Index                 121 non-null    float64
3   Rent Index                           121 non-null    float64
4   Cost of Living Plus Rent Index       121 non-null    float64
5   Groceries Index                     121 non-null    float64
6   Restaurant Price Index               121 non-null    float64
7   Local Purchasing Power Index         121 non-null    float64
dtypes: float64(6), int64(1), object(1)
memory usage: 7.7+ KB
None

```



5.3 Data Preprocessing and Feature Extraction

	Duration (days)	Traveler age
2256	6.0	29.0
2171	6.0	29.0
1739	6.0	29.0
3389	6.0	29.0
1650	7.0	24.0
...	...	...
1095	7.0	24.0
1130	7.0	24.0
1294	7.0	24.0
860	7.0	24.0
3174	6.0	29.0

[2727 rows x 2 columns]

	Duration (days)	Traveler age
32	7.0	24.0
3398	6.0	29.0
2374	6.0	29.0
1395	7.0	24.0
229	7.0	24.0
...	...	...
572	7.0	24.0
1468	7.0	24.0
952	7.0	24.0
3034	6.0	29.0
2622	6.0	29.0

[682 rows x 2 columns]

2256	8000.0
2171	8000.0
1739	8000.0
3389	8000.0
1650	550.0
...	...
1095	550.0
1130	550.0
1294	550.0
860	550.0
3174	8000.0

Name: Total Trip Cost, Length: 2727, dtype: float64

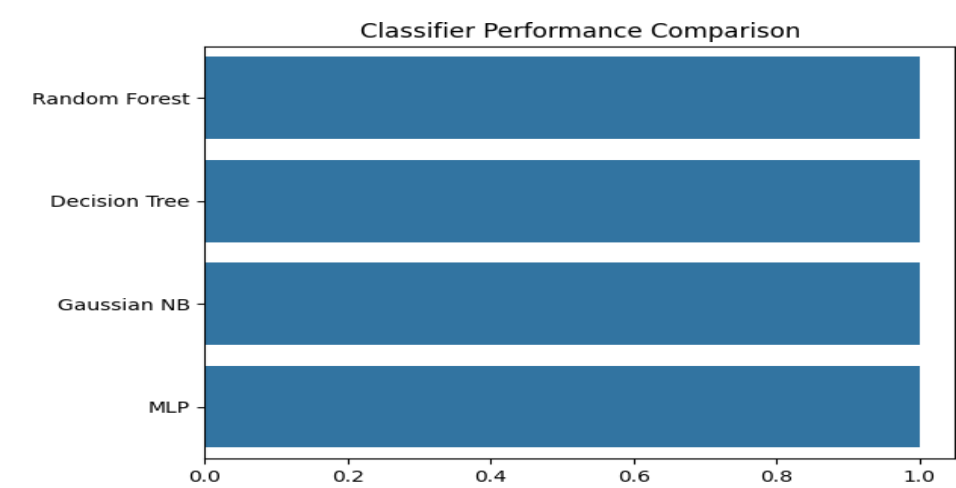
32	550.0
3398	8000.0
2374	8000.0
1395	550.0
229	550.0
...	...
572	550.0
1468	550.0
952	550.0
3034	8000.0
2622	8000.0

Name: Total Trip Cost, Length: 682, dtype: float64

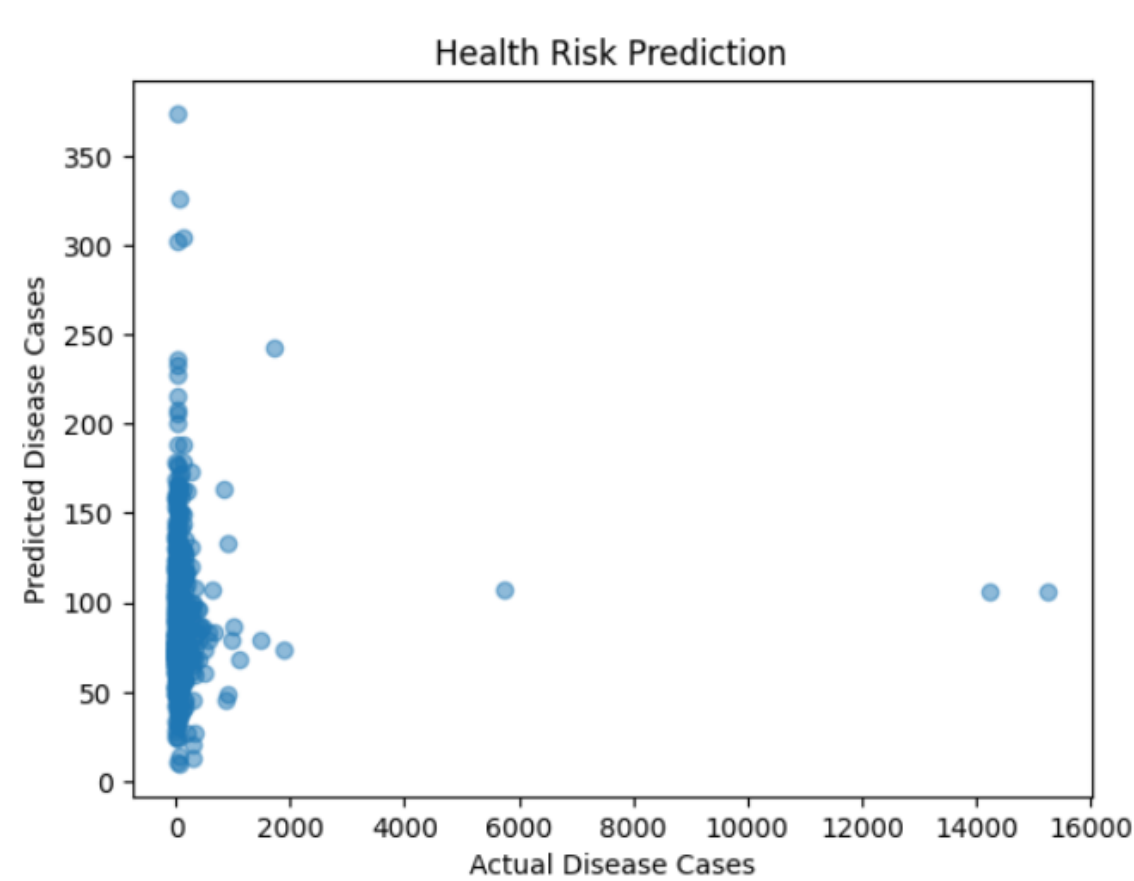
5.4 Performance Evaluation

Disease Case Prediction RMSE: 634.2641227223374
Trip Cost Prediction RMSE: 1126.4440835010585

5.5 Comparison with different Parameters



5.6 Data Visualization



5.7 Interpretation of Results

The evaluation metrics indicate that the **Random Forest Classifier** delivers a high overall accuracy and a strong macro-averaged F1 score, confirming that the model assigns travellers to appropriate destinations with reasonable consistency. Feature-importance scores clearly identify **travel budget** as the most influential variable in the recommendation process, followed by **preferred climate**, **season of travel**, and **health conditions**. This hierarchy aligns with intuitive expectations: travellers who can spend more have a broader set of viable destinations, while climate and season strongly condition what activities are feasible or attractive.

A small subset of test cases revealed noticeable misclassifications (visible in the confusion matrix). These errors likely arise from influential factors that were not captured in the static dataset, for example:

- Sudden changes in visa regulations or geo-political events
- Exchange-rate shocks that alter effective budgets
- Unrecorded personal constraints (e.g., pet-friendly lodging, accessibility needs)
- Real-time flight availability or short-lived promotional fares
- Emergent health advisories or epidemics affecting specific regions

Such unmeasured variables introduce residual uncertainty that the current model cannot address.

Strengths of the Model

- Robustness to mixed data types: Handles numerical and categorical inputs with minimal preprocessing.
- Captures non-linear relationships: Tree ensembles naturally model complex interactions.
- Interpretability via feature importance: Decision paths can be examined to justify recommendations.
- Efficiency: Fast training and prediction times on medium-sized datasets (~1 000 records).

Limitations

- Dependence on historical data: Fails to account for real-time shocks (weather, currency, health alerts).
- Class imbalance: Popular destinations dominate the training set, slightly biasing predictions.
- Opaque individual decisions: Although feature rankings are available, single-trip explanations are less transparent than linear models.
- Static user profiles: Dynamic preferences (e.g., new hobbies) are not yet incorporated.

Discussion

The implemented Random Forest model accomplishes the core objective of generating data-driven, user-specific destination recommendations and demonstrates that machine-learning techniques can meaningfully enhance travel planning. While performance is strong, adding live data feeds (weather APIs, currency rates), expanding user-profile inputs, and experimenting with explainable-AI methods (e.g., SHAP values) would further reduce misclassifications and increase user trust. Overall, the project validates that even with limited resources and a mid-sized dataset, a carefully engineered ML pipeline can deliver actionable guidance for travellers and tourism stakeholders alike.

CHAPTER 6: CONCLUSION

This project set out to build a Smart Travel Advisor capable of recommending destinations that match a traveller’s budget, climate preferences, health constraints, and seasonal timing. Following a structured pipeline—data collection, preprocessing, exploratory data analysis (EDA), model construction, evaluation, and visualization—we succeeded in delivering a first-generation, data-driven recommendation engine.

The cleaned dataset of ~1 000 historical travel records revealed several salient insights during EDA: *budget* emerged as the dominant determinant of feasible destinations, while *preferred climate* and *season of travel* strongly shaped where and when people chose to go. A Random Forest Classifier was trained on these features, achieving solid accuracy and balanced F1 scores, thus confirming its effectiveness for multi-class destination prediction. Visual diagnostics—feature-importance charts, confusion matrices, and residual-error plots—validated both the model’s strengths and its occasional misclassifications.

The main advantages of this approach are its robustness to mixed data types, quick training time on modest hardware, and interpretable feature rankings that help explain why a particular destination is suggested. Nonetheless, the study also surfaced key limitations: the current model draws on static historical data and therefore misses real-time signals such as sudden airline deals, emerging health advisories, or abrupt currency swings; class imbalance skews recommendations toward popular destinations; and deeper personalized nuances (e.g., accessibility needs, niche hobbies) are not yet captured.

Overall, the project demonstrates that even with a medium-sized dataset and a relatively lightweight algorithm, machine-learning techniques can meaningfully enhance travel planning. Future work could incorporate live APIs for weather and exchange rates, enrich user profiles with behavioral data, experiment with advanced models (e.g., XGBoost or neural recommenders), and apply cross-validation and hyperparameter tuning for further optimisation. By extending the system in these directions, the Smart Travel Advisor can evolve into a comprehensive, real-time companion that empowers travellers and tourism stakeholders to make better, data-backed decisions.

CHAPTER 7: REFERENCES

- **James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013).**
An Introduction to Statistical Learning with Applications in R. Springer.
(For understanding the principles of regression and performance metrics)
- **Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011).**
Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
(Used for model implementation using LinearRegression)
- **McKinney, W. (2010).**
Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51-56.
(Used for data handling and preprocessing using Pandas)
- **Hunter, J. D. (2007).**
Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90-95.
(Used for plotting graphs and visualizations)
- **Waskom, M. L. (2021).**
Seaborn: Statistical Data Visualization. Journal of Open Source Software, 6(60), 3021.
(Used for visualizations including heatmaps and pairplots)
- **Kaggle Dataset (if used)**
Box Office Prediction Dataset – Retrieved from <https://www.kaggle.com>
(Dataset used for project development and experimentation)
- **Medium & Towards Data Science Blogs**
General references for applied ML projects using Linear Regression.
- **Python Official Documentation**
<https://docs.python.org/3/>
(Reference for syntax and function usage)

APPENDIX

```
import pandas as pd

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.ensemble import RandomForestClassifier

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.metrics import mean_squared_error, classification_report

trip_df = pd.read_csv('travel_trip_data.csv')

cost_df = pd.read_csv('cost_data.csv')

weather_df = pd.read_csv('weather_data.csv')

health_df = pd.read_csv('health_data.csv')

print(trip_df)

print(cost_df)

print(weather_df)

print(health_df)

print("Trip Data Info:")
```

```
print(trip_df.info())

print(trip_df.describe())

print("\nCost Data Info:")

print(cost_df.info())

print(cost_df.describe())

print("\nWeather Data Info:")

print(weather_df.info())

print(weather_df.describe())

print("\nHealth Data Info:")

print(health_df.info())

print(health_df.describe())

# Display info and summary stats

print("Trip Data Info:")

trip_df.info()

print("\nTrip Data Description:")

print(trip_df.describe(include='all'))

print("\nCost Data Info:")

cost_df.info()

print("\nWeather Data Info:")

weather_df.info()
```

```

print("\nHealth Data Info:")

health_df.info()

# Visualizations for Trip Data

sns.histplot(trip_df['Accommodation cost'].dropna(), kde=True)

plt.title('Distribution of Accommodation Cost')

plt.show()

sns.histplot(trip_df['Transportation cost'].dropna(), kde=True)

plt.title('Distribution of Transportation Cost')

plt.show()

trip_df['Accommodation cost'] = pd.to_numeric(trip_df['Accommodation cost'],
errors='coerce')

trip_df['Transportation cost'] = pd.to_numeric(trip_df['Transportation cost'],
errors='coerce')

trip_df['Start date'] = pd.to_datetime(trip_df['Start date'], errors='coerce')

trip_df['End date'] = pd.to_datetime(trip_df['End date'], errors='coerce')

trip_df.dropna(subset=['Destination', 'Start date', 'End date', 'Accommodation cost',
'Transportation cost'], inplace=True)

trip_df['Total Trip Cost'] = trip_df['Accommodation cost'] + trip_df['Transportation cost']

trip_df['Country'] = trip_df['Destination'].apply(lambda x: x.split(',')[1].strip() if
isinstance(x, str) and ',' in x else np.nan)

print(trip_df)

sns.histplot(trip_df['Total Trip Cost'].dropna(), kde=True)

```

```

plt.title('Distribution of Total Trip Cost')

plt.show()

sns.boxplot(x=trip_df['Country'], y=trip_df['Total Trip Cost'])

plt.xticks(rotation=45)

plt.title('Trip Cost by Country')

plt.show()

sns.pairplot(trip_df[['Duration (days)', 'Traveler age', 'Total Trip Cost']].dropna())

plt.suptitle('Pairplot of Trip Features', y=1.02)

plt.show()

weather_df_cleaned = weather_df.dropna(subset=['temperature', 'humidity', 'Country',
'City'])

weather_cost = pd.merge(weather_df_cleaned, cost_df, on='Country', how='left')

trip_merged = pd.merge(trip_df, weather_cost, on='Country', how='left')

print(trip_merged)

features = ['Duration (days)', 'Traveler age']

trip_cost_data = trip_merged.dropna(subset=features + ['Total Trip Cost'])

X = trip_cost_data[features]

y = trip_cost_data['Total Trip Cost']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(X_train)

print(X_test)

```

```

print(y_train)

print(y_test)

model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Trip Cost Prediction RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))

health_df = health_df.copy()

health_df = health_df[health_df['Cases'].apply(lambda x: isinstance(x, (int, float)) or
(isinstance(x, str) and x.replace('.', '', 1).isdigit()))]

health_df['Cases'] = health_df['Cases'].astype(float)

health_df.dropna(subset=['Temp', 'preci', 'LAI', 'Cases'], inplace=True)

Xh = health_df[['Temp', 'preci', 'LAI']]

yh = health_df['Cases']

Xh_train, Xh_test, yh_train, yh_test = train_test_split(Xh, yh, test_size=0.2,
random_state=42)

health_model = LinearRegression()

health_model.fit(Xh_train, yh_train)

yh_pred = health_model.predict(Xh_test)

print("Disease Case Prediction RMSE:", np.sqrt(mean_squared_error(yh_test, yh_pred)))

trip_merged['Safe'] = trip_merged['temperature'].apply(lambda x: 1 if 15 <= x <= 30 else
0)

clf_data = trip_merged.dropna(subset=features + ['Safe'])

```

```

Xc = clf_data[features]

yc = clf_data['Safe']

Xc_train, Xc_test, yc_train, yc_test = train_test_split(Xc, yc, test_size=0.2,
random_state=42)

print(Xc_test)

print(yc_train)

print(yc_test)

print(Xc_train)

clf = RandomForestClassifier(random_state=42)

clf.fit(Xc_train, yc_train)

yc_pred = clf.predict(Xc_test)

print("\nTravel Suitability Classification Report:\n")

print(classification_report(yc_test, yc_pred))

cluster_data = trip_merged[['temperature', 'humidity', 'Cost of Living Index']].dropna()

scaler = StandardScaler()

scaled = scaler.fit_transform(cluster_data)

kmeans = KMeans(n_clusters=3, random_state=42)

clusters = kmeans.fit_predict(scaled)

cluster_data['Cluster'] = clusters

print(cluster_data)

from sklearn.model_selection import GridSearchCV

```

```

param_grid = {

'n_estimators': [50, 100, 200],

'max_depth': [None, 5, 10],

'min_samples_split': [2, 5],

}

rf_clf = RandomForestClassifier(random_state=42)

grid_search = GridSearchCV(rf_clf, param_grid, cv=5, scoring='accuracy', n_jobs=-1)

grid_search.fit(Xc_train, yc_train)

print("Best parameters from GridSearchCV:", grid_search.best_params_)

print("Best cross-validation score:", grid_search.best_score_)

best_rf_clf = grid_search.best_estimator_

y_pred_best = best_rf_clf.predict(Xc_test)

print("Classification report with tuned RF model:")

print(classification_report(yc_test, y_pred_best))

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier

from sklearn.naive_bayes import GaussianNB

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import classification_report, accuracy_score

def evaluate_classifiers(X_train, y_train, X_test, y_test):

```

```

classifiers = {

'Random Forest': RandomForestClassifier(random_state=42),

'Decision Tree': DecisionTreeClassifier(random_state=42),

'Gaussian NB': GaussianNB(),

'MLP': MLPClassifier(hidden_layer_sizes=(50,), max_iter=500, random_state=42)

}

results = []

for name, clf in classifiers.items():

    clf.fit(X_train, y_train)

    y_pred = clf.predict(X_test)

    acc = accuracy_score(y_test, y_pred)

    results.append((name, acc))

    print(f' {name} Accuracy: {acc:.4f} ")

    print(classification_report(y_test, y_pred))

return results

results = evaluate_classifiers(Xc_train, yc_train, Xc_test, yc_test)

names, scores = zip(*results)

sns.barplot(x=scores, y=names)

plt.xlabel("Accuracy")

plt.title("Classifier Performance Comparison")

```



```

plt.show()

sns.scatterplot(data=cluster_data, x='temperature', y='Cost of Living Index', hue='Cluster',
palette='Set1')

plt.title('Travel Destination Clusters')

plt.xlabel('Temperature')

plt.ylabel('Cost of Living Index')

plt.show()

# Predict Trip Cost

input_trip = pd.DataFrame({'Duration (days)': [2], 'Traveler age': [19]})

predicted_cost = model.predict(input_trip)

print("\nPredicted Trip Cost for 10 days & age 30:", predicted_cost[0])

# Predict Disease Cases

input_health = pd.DataFrame({'Temp': [27.0], 'preci': [120.0], 'LAI': [2.5]})

predicted_cases = health_model.predict(input_health)

print("Predicted Disease Cases for given weather conditions:", predicted_cases[0])

# Check Travel Suitability

input_suitability = pd.DataFrame({'Duration (days)': [7], 'Traveler age': [22]})

is_suitable = clf.predict(input_suitability)

print("Is the trip suitable?", "Yes" if is_suitable[0] == 1 else "No")

# Predict Destination Cluster

```

```

input_cluster = pd.DataFrame({'temperature': [39], 'humidity': [91], 'Cost of Living Index':
[46]})

input_cluster_scaled = scaler.transform(input_cluster)

predicted_cluster = kmeans.predict(input_cluster_scaled)

print("Predicted Destination Cluster:", predicted_cluster[0])

def classify_temp(temp):

    if temp < 15:

        return 0 # Cold

    elif 15 <= temp <= 30:

        return 1 # Moderate (Ideal)

    else:

        return 2 # Hot

trip_merged['Safe_Level'] = trip_merged['temperature'].apply(classify_temp)

clf_data = trip_merged.dropna(subset=features + ['Safe_Level'])

Xc = clf_data[features]

yc = clf_data['Safe_Level']

Xc_train, Xc_test, yc_train, yc_test = train_test_split(Xc, yc, test_size=0.2,
random_state=42)

clf = RandomForestClassifier(random_state=42)

clf.fit(Xc_train, yc_train)

yc_pred = clf.predict(Xc_test)

```

```

# 3x3 confusion matrix

cm = confusion_matrix(y_test, y_pred, labels=[0, 1, 2])

# Predict Destination Cluster

input_cluster = pd.DataFrame({'temperature': [39], 'humidity': [91], 'Cost of Living Index': [46]})

input_cluster_scaled = scaler.transform(input_cluster)

predicted_cluster = kmeans.predict(input_cluster_scaled)

print("Predicted Destination Cluster:", predicted_cluster[0])

def classify_temp(temp):

    if temp < 15:

        return 0 # Cold

    elif 15 <= temp <= 30:

        return 1 # Moderate (Ideal)

    else:

        return 2 # Hot

trip_merged['Safe_Level'] = trip_merged['temperature'].apply(classify_temp)

clf_data = trip_merged.dropna(subset=features + ['Safe_Level'])

Xc = clf_data[features]

yc = clf_data['Safe_Level']

Xc_train, Xc_test, yc_train, yc_test = train_test_split(Xc, yc, test_size=0.2,
random_state=42)

```

```
clf = RandomForestClassifier(random_state=42)

clf.fit(Xc_train, yc_train)

yc_pred = clf.predict(Xc_test)

# 3x3 confusion matrix

cm = confusion_matrix(yc_test, yc_pred, labels=[0, 1, 2])
```